

1 Eine Berechnung - ohne Rechnereinsatz

Im Folgenden wird ohne Rechnereinsatz $P_A(200, 001)$, $P_B(200, 001)$ und $P_U(200, 001)$ rekursiv berechnet.

Ausgangspunkt ist die folgende Gleichung:

$$P(200, 001) = p_1 P(100, 001) + p_2 P(200, 001) + p_3 P(200, 000)$$

Nun muss so lange „gerechnet“ werden, bis das erste Argument 000 oder das zweite Argument 000 ergibt. Wenn beide 000 werden, endet das Spiel unentschieden.

$$P(200, 001)(1 - p_2) = p_1 P(100, 001) + p_3 P(200, 000);$$

$$P(200, 001) = \frac{p_1}{p_1 + p_3} P(100, 001) + \frac{p_3}{p_1 + p_3} P(200, 000)$$

$P(200, 000)$ bedeutet, dass A noch zwei Chips auf dem ersten Feld hat und B alle Chips abgeräumt und somit gewonnen hat. Somit gilt

$$P_A(200, 000) = 0 \quad \text{und} \quad P_B(200, 000) = 1.$$

$$P(100, 001) = p_1 P(000, 001) + p_2 P(100, 001) + p_3 P(100, 000)$$

$$P(100, 001)(1 - p_2) = p_1 P(000, 001) + p_3 P(100, 000)$$

$$P(100, 001) = \frac{p_1}{p_1 + p_3} P(000, 001) + \frac{p_3}{p_1 + p_3} P(100, 000)$$

Damit folgt:

$$P(200, 001) = \left(\frac{p_1}{p_1 + p_3} \right)^2 P(000, 001) + \frac{p_1 \cdot p_3}{(p_1 + p_3)^2} P(100, 000) + \frac{p_3}{p_1 + p_3} P(200, 000)$$

Der erste Term liefert P_A , da A seine Chips als erster abgeräumt hat (000), die anderen Terme zeigen, dass B gewonnen hat. Es gilt somit:

$$P_A(200, 001) = \left(\frac{p_1}{p_1 + p_3} \right)^2$$

$$\begin{aligned} P_B(200, 001) &= \frac{p_1}{p_1 + p_3} \cdot \frac{p_3}{p_1 + p_3} + \frac{p_3}{p_1 + p_3} \\ &= p_3 \cdot \frac{2p_1 + p_3}{(p_1 + p_3)^2}. \end{aligned}$$

$$P_U(200, 001) = 0$$

Damit A gewinnt, muss A zweimal die beiden Chips auf dem ersten Feld abräumen, bevor B den einzigen Chip auf Feld 3 abräumen könnte.

Es gilt $P(A \text{ vor } B) = \frac{p_1}{p_1 + p_3}$ bzw. $P(B \text{ vor } A) = \frac{p_2}{p_1 + p_3}$.

Damit B gewinnt, muss A einmal vor B einen Chip auf Feld 1 abräumen und danach B vor A , oder B räumt vor A den einzigen Chip auf Feld 3 ab und hat wie im ersten Fall gewonnen.

$$P_B(200, 001) = \frac{p_1}{p_1 + p_3} \cdot \frac{p_3}{p_1 + p_3} + \frac{p_3}{p_1 + p_3}$$

ab.

2 Ein Chat mit ChatGPT

Das Rennen „A vor B um das erste Abräumen“ lässt sich äußerst elegant über bedingte Wahrscheinlichkeiten fassen, ganz ohne Rekursion:

1. Definition der Ereignisse

- Sei E das Ereignis „in einer Runde räumt jemand ab“ – also „entweder A oder B entfernt einen Chip“.
 - Sei A das Teilereignis „A räumt ab“ und B das Teilereignis „B räumt ab“.
- In einer einzigen Runde gilt

$$P(A) = p_1, \quad P(B) = p_2, \quad P(\text{niemand}) = p_3,$$

wobei „niemand“ hier bedeutet, dass beide auf einem leeren Feld landen und deshalb nichts entfernt wird.

2. Bedingte Wahrscheinlichkeit für „A vor B“

Wir interessieren uns für die Wahrscheinlichkeit, dass **in der Runde, in der erstmals jemand abräumt**, dies A und nicht B ist. Das sind genau die Fälle, in denen

1. alle vorangehenden Runden „niemand entfernt etwas“ verliefen (jeweils mit Wahrscheinlichkeit p_3),
2. und dann in der entscheidenden Runde A abräumt (mit Wahrscheinlichkeit p_1),
3. **und nicht** B abräumt.

Formal summieren wir über alle möglichen Längen $k - 1$ „Leer“-Runden:

$$\begin{aligned} P(\text{A räumt vor B}) &= \sum_{k=1}^{\infty} P(\underbrace{\text{niemand}, \dots, \text{niemand}}_{k-1}) P(A) \\ &= \sum_{k=1}^{\infty} p_3^{k-1} p_1 = p_1 \sum_{k=0}^{\infty} p_3^k = p_1 \frac{1}{1 - p_3} = \frac{p_1}{1 - p_3}. \end{aligned}$$

Da $1 - p_2 = p_1 + p_3$, folgt

$$P(\text{A vor B}) = \frac{p_1}{p_1 + p_3}.$$

Analog gilt $P(\text{B vor A}) = p_3/(p_1 + p_3)$.

3. Übertragung auf unser Beispiel $S = (2, 0, 0)$, $T = (0, 0, 1)$

- Damit A das komplette Duell gewinnt (zwei Chips auf Feld 0 gegen einen auf Feld 2), muss A **zweimal hintereinander** vor B abräumen.
- Die beiden Rennen „erstes Chip-Abräumen“ sind voneinander unabhängig (nach dem ersten Abräumen startet der gleiche Mechanismus erneut), also

$$P_A(200, 001) = P(\text{A vor B})^2 = \left(\frac{p_1}{p_1 + p_3}\right)^2.$$

- Entsprechend ergibt sich für B

$$P_B(200, 001) = 1 - P_A(200, 001) = 1 - \left(\frac{p_1}{p_1 + p_3}\right)^2 = \frac{(p_1 + p_3)^2 - p_1^2}{(p_1 + p_3)^2} = \frac{2p_1 p_3 + p_3^2}{(p_1 + p_3)^2},$$

und natürlich $P_U = 0$.

4. Fazit

- **Bedingte Wahrscheinlichkeiten** erlauben eine sehr kompakte Herleitung von $P(\text{A vor B})$.
- **Unabhängigkeit der „Rennen“** (jeweils bis zum ersten Abräumen) liefert sofort das Quadrat für P_A .
- So gewinnt man ohne komplizierte Rekursion in ein paar Zeilen den gleichen eleganten Endausdruck.

Sehr schön dargestellt. In der Rekursion wird der unendliche Prozess dadurch sichtbar, dass auf der linken und rechten Seite "der gleiche Zustand" steht. Auf der linken Seite gewichtet mit p_2 . Zusammenfassen und dann dividieren liefert das Ergebnis. Ich finde die Vernetzungen schön.

Ganz genau – und gerade diese **Selbstreferenz** macht die Rekursion so anschaulich:

- Die Gleichung

$$P = p_2 P + (\text{Rest})$$

zeigt unmittelbar, dass sich die „Null“-Runden (jeweils mit Wahrscheinlichkeit p_2) selbst wiederholen, solange niemand abräumt.

- Durch Umstellen auf

$$P(1 - p_2) = (\text{Rest}) \implies P = \frac{(\text{Rest})}{1 - p_2}$$

hebt man diesen **unendlichen Zyklus** elegant heraus und erhält die normierte Formel.

Dieses Zusammenspiel aus **rekursiver Perspektive** (Selbstschleife) und **linearem Auflösen** (Division durch $1 - p_2$) ist wirklich ein schönes Beispiel dafür, wie in der Wahrscheinlichkeitsrechnung oft simple algebraische Tricks tiefe stochastische Prozesse auf einen eindeutigen geschlossenen Ausdruck reduzieren können.

3 Python-Listing zu symbolischen Berechnungen

Listing 1: Exakte Berechnungen - mit Variablen (04a-ein-Wuerfel-Fall-symbolisch.ipynb)

```

1  import sympy as sp
2  from functools import lru_cache
3
4  # Symbole f r Wahrscheinlichkeiten
5  p_1, p_2, p_3 = sp.symbols('p_1 p_2 p_3', nonnegative=True)
6  p = (p_1, p_2, p_3)
7
8  A = (2, 0, 0)
9  B = (0, 0, 1)
10
11 @lru_cache(None)
12 def P_A(V, W):
13     sum_V, sum_W = sum(V), sum(W)
14     if sum_V == 0 and sum_W == 0: return sp.Rational(0)
15     if sum_V == 0: return sp.Rational(1)
16     if sum_W == 0: return sp.Rational(0)
17
18     s = sum(pj for pj, vj, wj in zip(p, V, W) if vj or wj)
19     if s == 0: return sp.Rational(0)
20
21     prob = 0
22     for j, pj in enumerate(p):
23         if V[j] or W[j]:
24             Vn = V[:j] + (max(0, V[j]-1),) + V[j+1:]
25             Wn = W[:j] + (max(0, W[j]-1),) + W[j+1:]
26             prob += (pj / s) * P_A(Vn, Wn)
27     return sp.simplify(prob)
28
29 PA = sp.simplify(P_A(A, B))
30 PB = sp.simplify(P_A(B, A))
31 PU = sp.simplify(1 - PA - PB)
32
33 simplified_PA = sp.simplify(PA.subs(p_1 + p_2 + p_3, 1))
34 simplified_PB = sp.simplify(PB.subs(p_1 + p_2 + p_3, 1))
35 simplified_PU = sp.simplify(PU.subs(p_1 + p_2 + p_3, 1))
36
37 simplified_PA, simplified_PB, simplified_PU

```

Ausgabe:

$$(p_1 * 2 / (p_1 + p_3) * 2, p_3 * (2 * p_1 + p_3) / (p_1 + p_3) * 2, 0)$$

Der Programmcode wurde ohne weitere Hinweise an ChatGPT 5 übergeben, mit der Bitte um eine Dokumentation. Die folgenden Ausführungen wurden nicht mehr überarbeitet.

Überblick - erstellt von ChatGPT 5

Dieses Programm berechnet symbolisch Wahrscheinlichkeiten für ein zufälliges Auswahlverfahren. Gegeben sind zwei „Spieler“ (oder konkurrierende Ereignisse) A und B, die jeweils bestimmte Mengen an Objekten besitzen. Jedes Objekt gehört zu einer von drei Kategorien (Index 1–3), mit Wahrscheinlichkeiten p_1, p_2, p_3 .

Das Programm berechnet:

- $P(A)$: Wahrscheinlichkeit, dass A gewinnt,
- $P(B)$: Wahrscheinlichkeit, dass B gewinnt,
- $P(U)$: Wahrscheinlichkeit eines Unentschiedens.

Aufbau des Codes

1. Symbole für Wahrscheinlichkeiten

```
p_1, p_2, p_3 = sp.symbols('p_1 p_2 p_3', nonnegative=True)
p = (p_1, p_2, p_3)
```

Die drei Symbole p_1, p_2, p_3 stehen für Wahrscheinlichkeiten, die sich zu 1 summieren sollen. p ist ein Tupel, das diese Variablen gruppiert.

2. Anfangszustände

```
A = (2, 0, 0)
B = (0, 0, 1)
```

- Spieler A hat 2 Objekte der Kategorie 1,
- Spieler B hat 1 Objekt der Kategorie 3.

Allgemein ist ein Zustand wie (v_1, v_2, v_3) eine Verteilung von Objekten über die Kategorien.

3. Rekursive Wahrscheinlichkeit $P_A(V, W)$

```
@lru_cache(None)
def P_A(V, W):
    sum_V, sum_W = sum(V), sum(W)
    if sum_V == 0 and sum_W == 0: return sp.Rational(0)
    if sum_V == 0: return sp.Rational(1)
    if sum_W == 0: return sp.Rational(0)
```

Eingabe: zwei Zustände V und W (jeweils ein Tupel von Anzahlen).

Ziel: Berechne die Wahrscheinlichkeit, dass Spieler A gewinnt, beginnend bei diesen Zuständen.

Abbruchbedingungen:

- Wenn beide keine Objekte mehr haben $\rightarrow 0$ (niemand gewinnt),
- Wenn A keine Objekte mehr hat $\rightarrow 1$ (B gewinnt),
- Wenn B keine Objekte mehr hat $\rightarrow 0$ (A verliert nicht).

4. Wahrscheinlichkeitsübergang

```
s = sum(pj for pj, vj, wj in zip(p, V, W) if vj or wj)
if s == 0: return sp.Rational(0)
```

s ist die Gesamtsumme der Wahrscheinlichkeiten über alle Kategorien, in denen A oder B noch Objekte haben.

5. Rekursion

```

prob = 0
for j, pj in enumerate(p):
    if V[j] or W[j]:
        Vn = V[:j] + (max(0, V[j]-1),) + V[j+1:]
        Wn = W[:j] + (max(0, W[j]-1),) + W[j+1:]
        prob += (pj / s) * P_A(Vn, Wn)

```

Für jede Kategorie j :

- Falls A oder B dort Objekte hat, wird eines entfernt,
- Mit Wahrscheinlichkeit $\frac{p_j}{s}$ geht das Spiel in den reduzierten Zustand (Vn, Wn) ,
- P_A wird rekursiv aufgerufen, bis ein Endzustand erreicht wird.

6. Berechnung und Vereinfachung

```

PA = sp.simplify(P_A(A, B))
PB = sp.simplify(P_A(B, A))
PU = sp.simplify(1 - PA - PB)

```

- P_A : Wahrscheinlichkeit, dass A gewinnt,
- P_B : Wahrscheinlichkeit, dass B gewinnt,
- P_U : Restwahrscheinlichkeit (Unentschieden).

```

simplified_PA = sp.simplify(PA.subs(p_1 + p_2 + p_3, 1))
simplified_PB = sp.simplify(PB.subs(p_1 + p_2 + p_3, 1))
simplified_PU = sp.simplify(PU.subs(p_1 + p_2 + p_3, 1))

```

Da sich die Wahrscheinlichkeiten zu 1 summieren, wird die Bedingung $p_1 + p_2 + p_3 = 1$ eingesetzt.

Ergebnis

Das Programm liefert symbolische Ausdrücke für

$$P(A), \quad P(B), \quad P(U).$$

Fazit

- Das Programm implementiert ein rekursives, stochastisches Auswahlmodell,
- Es nutzt Memoization (`lru_cache`) zur Beschleunigung,
- Mit Sympy werden die Ergebnisse symbolisch vereinfacht.